

Audit Log Management and Visualization

Lecture by Assist. Prof. Dr. Chanankorn Jandaeng

Walailak University

Duration: 90 minutes

Objective: Understand the purpose, design, and technology of audit logs, their security role, and demonstrate a basic ReactJS dashboard for visualization.

Lecture Flow

1. What is an Audit Log?
2. Why are Audit Logs Important?
3. How Audit Logs Work
4. Log Standards and Formats
5. Log Management Technologies
6. Challenges in Log Handling
7. Demonstration: ReactJS Log Dashboard
8. Summary & Best Practices

1. What is an Audit Log?

Definition:

An **audit log** (or audit trail) is a **chronological record of system activities** that provides evidence of actions performed by users, systems, or processes.

Purpose:

To ensure **accountability, traceability, and integrity** of operations within IT systems.

1. What is an Audit Log?

Typical Data Elements:

Field	Description
Timestamp	Exact time of event
User / Process	Actor or initiator
Action	Operation performed
Object	Target resource (file, record, endpoint)
Result	Success / Failure
Source IP	Origin of request

Example Audit Log Entry

```
{  
  "timestamp": "2025-11-06T10:42:15Z",  
  "user": "admin",  
  "action": "DELETE /api/v1/resource/42",  
  "status": "success",  
  "source_ip": "192.168.10.24"  
}
```

- ✓ Represents a traceable system event.
- ✓ Provides accountability and forensic trace.

2. Why are Audit Logs Important?

Security Perspective

- Detect unauthorized access or privilege abuse
- Support **forensic investigation** post-incident
- Enable **intrusion detection** and **threat hunting**
- Demonstrate **compliance** with legal standards

Organizational Perspective

- Ensure transparency and accountability
- Provide operational visibility
- Improve troubleshooting and performance monitoring

Example Use Cases

Scenario	Audit Log Function
Data breach	Identify who accessed sensitive files
Configuration change	Verify legitimate modification
Legal compliance	Demonstrate adherence to ISO 27001 or GDPR
Insider threat	Detect privilege misuse
Forensics	Reconstruct timeline of events

3. How Audit Logs Work

Log Flow Process

[System or App] → [Log Collector] → [Parser & Normalizer] → [Storage/Indexing] → [Dashboard & Alerting]

Key Operations

1. **Capture:** System generates event data
2. **Normalize:** Convert to common schema (JSON recommended)
3. **Store:** Indexed in searchable database (Elasticsearch, PostgreSQL, etc.)
4. **Analyze:** Visualized or correlated for anomalies
5. **Retain:** Keep logs securely for audit duration (e.g., 1–2 years)

Example Log Generation (Application)

```
import logging, json
logging.basicConfig(filename='audit.log', level=logging.INFO)

def audit_event(user, action, result, ip):
    event = {
        "timestamp": "2025-11-06T10:50:00Z",
        "user": user,
        "action": action,
        "result": result,
        "ip": ip
    }
    logging.info(json.dumps(event))

audit_event("admin", "login", "success", "192.168.1.10")
```

✓ Each event recorded as structured JSON entry for analysis.

4. Log Standards and Formats

Common Log Standards

Standard	Description	Example Use
RFC 5424 (Syslog)	Structured log messaging standard	Network & system logs
Common Event Format (CEF)	Vendor-neutral format by ArcSight	Security event logs
LEEF	IBM's Lightweight Event Format	QRadar SIEM
JSON / GELF (Graylog)	Readable structured logging	Modern applications
Auditd (Linux)	Kernel-level audit framework	OS-level activity tracking

Example: Syslog vs. JSON

Syslog

```
<34>1 2025-11-06T10:50:00Z host app - ID47 [meta sequenceId="1234"] login success
```

JSON

```
{  
  "time": "2025-11-06T10:50:00Z",  
  "host": "app",  
  "event": "login",  
  "status": "success"  
}
```

✅ JSON preferred for modern log pipelines and visualization.

5. Log Management Technologies

Layer	Technologies	Function
Collection	Syslog, Filebeat, Fluentd, Logstash	Gather raw logs
Parsing / Normalization	Grok, Regex, JSON decode	Standardize formats
Storage	Elasticsearch, PostgreSQL, MongoDB	Retain & index
Analysis	Splunk, Kibana, Grafana	Search & visualize
Alerting	Wazuh, ELK, custom scripts	Notify anomalies

Trend: Cloud-native logging (AWS CloudWatch, GCP Logging, Azure Monitor)

6. Challenges in Log Management

Challenge	Description
Volume	Billions of events daily in enterprise networks
Velocity	Real-time log ingestion and processing
Variety	Different formats, sources, and standards
Integrity	Logs must be tamper-proof and trusted
Retention	Compliance with data storage laws
Correlation	Linking multiple log sources for incident context
Visualization	Making complex logs human-understandable

Solution: Automation, structured formats, and visual analytics.

7. Demonstration: API Audit Log & ReactJS Dashboard

Objective: Build a small full-stack simulation of real-world audit log collection.

System Architecture



```
graph LR; A[Application/API] --> B[FastAPI endpoint]; B --> C[JSON logs]; B --> D[ReactJS Dashboard];
```

[Application/API] → [FastAPI endpoint] → [JSON logs]
↓
[ReactJS Dashboard]

- **Backend:** Python FastAPI generates logs dynamically.
- **Frontend:** ReactJS fetches and visualizes audit log entries.

Backend Example (FastAPI)

```
# pip install fastapi uvicorn
from fastapi import FastAPI
from datetime import datetime
import random

app = FastAPI()
logs = []

@app.post("/generate")
def generate_log():
    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "user": random.choice(["admin", "analyst", "guest"]),
        "action": random.choice(["login", "update", "delete", "view"]),
        "result": random.choice(["success", "failed"]),
        "ip": f"192.168.1.{random.randint(1, 255)}"
    }
    logs.append(event)
    return {"message": "Log generated", "event": event}
```


Backend Example (FastAPI)

```
@app.get("/logs")
def get_logs():
    return {"data": logs}
```

Run with:

```
uvicorn main:app --reload
```

Frontend Example (ReactJS Dashboard)

```
import React, { useEffect, useState } from "react";
import axios from "axios";

export default function AuditDashboard() {
  const [logs, setLogs] = useState([]);

  useEffect(() => {
    const fetchLogs = async () => {
      const res = await axios.get("http://localhost:8000/logs");
      setLogs(res.data.data);
    };
    const timer = setInterval(fetchLogs, 2000);
    return () => clearInterval(timer);
  }, []);
```

Frontend Example (ReactJS Dashboard)

```
return (  
  <div className="p-6">  
    <h2 className="text-2xl font-bold mb-4">Audit Log Dashboard</h2>  
    <table className="table-auto w-full border">  
      <thead className="bg-gray-100">  
        <tr>  
          <th>Timestamp</th>  
          <th>User</th>  
          <th>Action</th>  
          <th>Result</th>  
          <th>IP</th>  
        </tr>  
      </thead>
```

Frontend Example (ReactJS Dashboard)

```
    <tbody>
      {logs.map((log, index) => (
        <tr key={index}>
          <td>{log.timestamp}</td>
          <td>{log.user}</td>
          <td>{log.action}</td>
          <td>{log.result}</td>
          <td>{log.ip}</td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
);
}
```

- ✓ Auto-refreshes logs via API.
- ✓ Provides real-time audit monitoring view.

Visualization Extension

Add charting with **Chart.js** or **Recharts** to show:

- Event frequency over time
- Distribution of actions (login, delete, view)
- Success vs. failed operations

```
import { Bar } from "react-chartjs-2";

const data = {
  labels: ["Login", "Update", "Delete", "View"],
  datasets: [{
    label: "Action Frequency",
    data: [15, 8, 5, 12],
    backgroundColor: "rgba(54, 162, 235, 0.6)"
  }]
};

<Bar data={data} />
```

8. Summary & Best Practices

Principle	Recommendation
What	Audit logs record system activities for accountability
Why	Critical for detection, forensics, and compliance
How	Use structured JSON logs, centralized storage, and secure retention
Standard	Follow Syslog / JSON / CEF with timestamps
Technology	Combine FastAPI (log API) + ReactJS (dashboard)
Challenge	High volume, correlation complexity, visualization
Practice	Automate log ingestion and protect integrity (hashing, RBAC)

End of Lecture

Assist. Prof. Dr. Chanankorn Jandaeng

Walailak University

 chatchanan.ja@mail.wu.ac.th

 Lecture Highlights

Area	Key Learning Outcome
Conceptual Foundation	Understood the What, Why, and How of audit logs — their role in accountability, traceability, and forensic readiness.
Security Relevance	Recognized audit logs as core evidence in incident response, regulatory compliance, and threat detection .
Standards & Formats	Learned structured log formats (Syslog RFC 5424, JSON, CEF) and the importance of timestamped, normalized events .
Technological Layers	Explored end-to-end log pipeline: collection → parsing → storage → indexing → visualization .



Lecture Highlights

Area	Key Learning Outcome
Challenges	Identified key issues — log volume , variety , integrity , and correlation complexity — with modern counter-solutions.
Practical Demonstration	Implemented a mini FastAPI log generator and ReactJS dashboard for real-time log visualization.
Visualization Skills	Applied ReactJS + Chart.js/Recharts to analyze log frequency and event distributions.
Professional Competency	Acquired fundamental skills to build or extend SIEM-like dashboards for SOC or research environments.